

Studentu pazymiu skaiciavimo programa

Generated by Doxygen 1.17.0

1 Studentu pazymiu skaiciavimo programa	1
1.1 Reikalavimai	1
1.2 Programos atsisiuntimas	1
1.3 Programos kompiliavimas su CMake	1
1.4 Programos paleidimas	1
1.5 Programos meniu	2
1.6 v3.0 nuosavas Vector konteineris	2
1.7 Realizuotos Vector funkcijos	2
1.8 Vector funkciju pavyzdziai	3
1.8.1 1. push_back()	3
1.8.2 2. at()	3
1.8.3 3. insert()	3
1.8.4 4. erase()	4
1.8.5 5. resize()	4
1.8.6 6. Iteratoriai ir std::sort()	4
1.9 push_back() spartos tyrimas	4
1.10 Atminties perskirstymu skaicius	4
1.11 Studentu programos tyrimas su std::vector ir Vector	4
1.12 Unit testai	5
1.13 Testavimo failai	6
1.14 Doxygen dokumentacija	6
1.15 Diegimo failas	7
1.16 Versiju aprasas	7
1.16.1 v3.0	7
1.16.2 v2.0	7
1.16.3 v1.5	8
1.16.4 v1.2	8
1.16.5 v1.1	8
1.16.6 v1.0	8
1.16.7 v0.4	8
1.16.8 v0.3	9
1.16.9 v0.2	9
1.16.10 v0.1	9
1.16.11 v.pradine	9
2 Directory Hierarchy	11
2.1 Directories	11
3 Hierarchical Index	13
3.1 Class Hierarchy	13
4 Class Index	15
4.1 Class List	15

5 File Index	17
5.1 File List	17
6 Directory Documentation	19
6.1 Objektinis Directory Reference	19
7 Class Documentation	21
7.1 Studentas Class Reference	21
7.1.1 Detailed Description	23
7.1.2 Constructor & Destructor Documentation	23
7.1.2.1 Studentas() [1/5]	23
7.1.2.2 Studentas() [2/5]	23
7.1.2.3 Studentas() [3/5]	23
7.1.2.4 ~Studentas()	23
7.1.2.5 Studentas() [4/5]	23
7.1.2.6 Studentas() [5/5]	24
7.1.3 Member Function Documentation	24
7.1.3.1 getEgzaminas()	24
7.1.3.2 getPazymiai()	24
7.1.3.3 getRezultatas()	24
7.1.3.4 getSunaikintuObjektuKiekis()	24
7.1.3.5 operator=() [1/2]	24
7.1.3.6 operator=() [2/2]	25
7.1.3.7 pridetiPazymi()	25
7.1.3.8 print()	25
7.1.3.9 read()	25
7.1.3.10 setEgzaminas()	26
7.1.3.11 setPazymiai()	26
7.1.3.12 setRezultatas()	26
7.1.4 Friends And Related Symbol Documentation	26
7.1.4.1 operator<<	26
7.1.4.2 operator>>	26
7.2 Vector< T > Class Template Reference	27
7.2.1 Detailed Description	29
7.2.2 Constructor & Destructor Documentation	29
7.2.2.1 Vector() [1/6]	29
7.2.2.2 Vector() [2/6]	29
7.2.2.3 Vector() [3/6]	30
7.2.2.4 Vector() [4/6]	30
7.2.2.5 ~Vector()	30
7.2.2.6 Vector() [5/6]	30
7.2.2.7 Vector() [6/6]	30
7.2.3 Member Function Documentation	31

7.2.3.1 assign() [1/2]	31
7.2.3.2 assign() [2/2]	31
7.2.3.3 at() [1/2]	31
7.2.3.4 at() [2/2]	31
7.2.3.5 back() [1/2]	31
7.2.3.6 back() [2/2]	31
7.2.3.7 begin() [1/2]	31
7.2.3.8 begin() [2/2]	32
7.2.3.9 capacity()	32
7.2.3.10 cbegin()	32
7.2.3.11 cend()	32
7.2.3.12 clear()	32
7.2.3.13 data() [1/2]	32
7.2.3.14 data() [2/2]	32
7.2.3.15 emplace()	32
7.2.3.16 emplace_back()	33
7.2.3.17 empty()	33
7.2.3.18 end() [1/2]	33
7.2.3.19 end() [2/2]	33
7.2.3.20 erase() [1/2]	33
7.2.3.21 erase() [2/2]	34
7.2.3.22 front() [1/2]	34
7.2.3.23 front() [2/2]	34
7.2.3.24 insert() [1/3]	34
7.2.3.25 insert() [2/3]	35
7.2.3.26 insert() [3/3]	35
7.2.3.27 max_size()	35
7.2.3.28 operator"!="()	35
7.2.3.29 operator=() [1/3]	36
7.2.3.30 operator=() [2/3]	36
7.2.3.31 operator=() [3/3]	36
7.2.3.32 operator==()	36
7.2.3.33 operator[]() [1/2]	37
7.2.3.34 operator[]() [2/2]	37
7.2.3.35 pop_back()	37
7.2.3.36 push_back() [1/2]	37
7.2.3.37 push_back() [2/2]	37
7.2.3.38 reserve()	37
7.2.3.39 resize() [1/2]	38
7.2.3.40 resize() [2/2]	38
7.2.3.41 shrink_to_fit()	38
7.2.3.42 size()	38

7.2.3.43 swap()	38
7.3 Zmogus Class Reference	38
7.3.1 Detailed Description	39
7.3.2 Constructor & Destructor Documentation	39
7.3.2.1 Zmogus() [1/2]	39
7.3.2.2 Zmogus() [2/2]	40
7.3.2.3 ~Zmogus()	40
7.3.3 Member Function Documentation	40
7.3.3.1 getPavarde()	40
7.3.3.2 getVardas()	40
7.3.3.3 print()	40
7.3.3.4 read()	40
7.3.3.5 setPavarde()	41
7.3.3.6 setVardas()	41
7.3.4 Member Data Documentation	41
7.3.4.1 pavarde_	41
7.3.4.2 vardas_	41
8 File Documentation	43
8.1 Objektinis/Studentas.h File Reference	43
8.1.1 Macro Definition Documentation	43
8.1.1.1 STUDENTAS_H	43
8.2 Studentas.h	43
8.3 Objektinis/Vector.h File Reference	44
8.3.1 Macro Definition Documentation	44
8.3.1.1 VECTOR_H	44
8.4 Vector.h	45
8.5 Objektinis/Zmogus.h File Reference	52
8.5.1 Macro Definition Documentation	52
8.5.1.1 ZMOGUS_H	52
8.6 Zmogus.h	52
8.7 README.md File Reference	53
Index	55

Chapter 1

Studentu pazymiu skaiciavimo programa

1.1 Reikalavimai

Norint paleisti programa, reikia tureti:

- C++ kompiliatoriu su C++17 palaikymu;
 - CMake;
 - Git;
 - Visual Studio, jeigu norima paleisti unit testus per `Test Explorer`;
 - Doxygen, jeigu norima sugeneruoti dokumentacija;
 - Inno Setup, jeigu norima sugeneruoti diegimo faila.
-

1.2 Programos atsisiuntimas

Projektas atsisiunciamas is GitHub:

```
git clone https://github.com/Boljerr/OOP_3.git
cd OOP_3
```

1.3 Programos kompiliavimas su CMake

Projektas kompiliuojamas naudojant CMake.

```
mkdir build
cd build
cmake ..
cmake --build . --config Release
```

Paleidziamas failas paprastai sukuriamas:

[build/Release/Studentai.exe](#)

1.4 Programos paleidimas

Is `build` aplanko:

```
./Release/Studentai.exe
```

Windows aplinkoje programa taip pat galima paleisti per Visual Studio arba tiesiog atidaryti sugeneruota `.exe` faila.

1.5 Programos meniu

```
1 - Rankinis ivedimas
2 - Generuoti tik pazymius
3 - Generuoti studentus ir pazymius
4 - Nuskaityti is failo
5 - Generuoti studentu faila
6 - Atlikti failo kurimo tyrima
7 - Atlikti v0.4 duomeniu apdorojimo tyrima(vector)
8 - Atlikti v1.1 konteineriu tyrima
9 - Atlikti v1.1 skirstymo strategiju tyrima
10 - Testuoti Studentas klase
11 - Atlikti v3.0 duomeniu apdorojimo tyrima(mano Vector)
12 - Atlikti std::vector ir Vector push_back tyrima
13 - Palyginti std::vector ir mano Vector su studentu failu
14 - Baigti
```

1.6 v3.0 nuosavas Vector konteineris

v3.0 versijoje sukurtas nuosavas sabloninis konteineris `Vector<T>`, kuris veikia panasiai kaip `std::vector`. Konteineris realizuotas naudojant dinamini masyva. Viduje saugomi trys pagrindiniai laukai:

- `T* duomenys_` - rodykle i dinamini masyva;
- `std::size_t dydis_` - dabartinis elementu kiekis;
- `std::size_t talpa_` - rezervuota vieta elementams.

Kai `push_back()` metu nebelieka vietos naujam elementui, konteinerio talpa padidinama ir seni elementai perkeliami i nauja masyva.

1.7 Realizuotos Vector funkcijos

Realizuotos pagrindines `std::vector` tipo funkcijos:

- `Vector()`
- `Vector(size_t kiekis)`
- `Vector(size_t kiekis, const T& reiksme)`
- `Vector(std::initializer_list<T>)`
- kopijavimo konstruktorius
- kopijavimo priskyrimo operatorius
- perkavimo konstruktorius
- perkavimo priskyrimo operatorius
- destruktorius
- `size()`
- `capacity()`
- `empty()`
- `max_size()`
- `operator[]`
- `at()`
- `front()`
- `back()`
- `data()`

- `begin()`
- `end()`
- `cbegin()`
- `cend()`
- `reserve()`
- `resize()`
- `shrink_to_fit()`
- `push_back()`
- `pop_back()`
- `clear()`
- `insert()`
- `erase()`
- `assign()`
- `swap()`
- `emplace_back()`
- `emplace()`
- `operator==`
- `operator!=`

1.8 Vector funkciju pavyzdziai

1.8.1 1. `push_back()`

```
Vector<int> v;  
  
v.push_back(1);  
v.push_back(2);  
v.push_back(3);
```

Rezultatas:

1 2 3

1.8.2 2. `at()`

```
Vector<int> v = { 4, 5, 6 };  
  
std::cout << v.at(1);
```

Rezultatas:

5

Jeigu indeksas yra uz ribu, `at()` meta `std::out_of_range` isimti.

1.8.3 3. `insert()`

```
Vector<int> v = { 1, 4 };  
  
v.insert(v.begin() + 1, { 2, 3 });
```

Rezultatas:

1 2 3 4

1.8.4 4. erase()

```
Vector<int> v = { 1, 2, 3, 4 };
```

```
v.erase(v.begin() + 1);
```

Rezultatas:

```
1 3 4
```

1.8.5 5. resize()

```
Vector<int> v = { 1, 2 };
```

```
v.resize(5, 9);
```

Rezultatas:

```
1 2 9 9 9
```

1.8.6 6. Iteratoriai ir std::sort()

```
Vector<int> v = { 3, 1, 2 };
```

```
std::sort(v.begin(), v.end());
```

Rezultatas:

```
1 2 3
```

1.9 push_back() spartos tyrimas

Buvo matuojama, kiek vidutiniškai laiko užtrunka tuscia `std::vector<int>` ir tuscia `Vector<int>` užpildyti naudojant `push_back()`.

Buvo atlikta 10 bandymu. Lenteleje pateikiamas vidurkis.

Elementu kiekis	std::vector vid. laikas, s	Vector vid. laikas, s
10 000	0.000178740	0.000053870
100 000	0.000558310	0.000396890
1 000 000	0.004688730	0.003114310
10 000 000	0.042939080	0.034631990
100 000 000	0.451623300	0.313787700

Siame teste mano `Vector` buvo greitesnis už `std::vector`, nes mano realizacijoje talpa didinama dvigubinant, todėl buvo mažiau atminties perskirstymu. `std::vector` talpos didinimo strategija priklauso nuo konkretios STL realizacijos.

1.10 Atminties perskirstymu skaičius

Perskirstymas buvo skaičiuojamas tada, kai prieš `push_back()` buvo tenkinama sąlyga:

```
capacity() == size()
```

Buvo užpildoma 100 000 000 int elementu.

Konteineris	Perskirstymu skaičius
std::vector	47
Vector	28

Mano `Vector` perskirstymu atliko mažiau, nes talpa buvo didinama dvigubinant.

1.11 Studentu programos tyrimas su std::vector ir Vector

Buvo palygintas studentu programos veikimas naudojant `std::vector` ir mano `Vector`.

Naudoti nustatymai:

- galutinis balas skaiciuotas pagal vidurki;
- studentai rusiuoti pagal galutini rezultata;
- naudotas skirstymas su `std::stable_partition`;
- kiekvienam failui atlikti 3 bandymai;
- lenteleje pateikiami laiku vidurkiai.

Studentu kiekis	Konteineris	Nuskaitymas, s	Skirstymas, s	Rusiavimas, s	Isvedimas, s	Bendras laikas, s
100 000	<code>std::vector</code>	0.499839	0.015915	0.021947	0.094470	0.633464
100 000	<code>Vector</code>	0.530764	0.018053	0.022191	0.093264	0.665642
1 000 000	<code>std::vector</code>	4.841230	0.152808	0.225936	0.856493	6.091330
1 000 000	<code>Vector</code>	4.954550	0.173366	0.223980	0.831624	6.199703
10 000 000	<code>std::vector</code>	49.436967	1.674317	2.427917	9.419157	63.118733
10 000 000	<code>Vector</code>	51.044167	1.422003	2.283997	8.322230	63.281300

Rezultatai rodo, kad studentu programoje `std::vector` ir mano `Vector` veike labai panasiai. `std::vector` buvo siek tiek greitesnis bendrame rezultate, nes tai standartines bibliotekos optimizuotas konteineris. Mano `Vector` veikia teisingai su tais paciais duomenimis, bet jo realizacija yra paprastesne.

1.12 Unit testai

Testams naudojamas **Visual Studio C++ Unit Test Framework**.

Testai pateikti faile:

`StudentasUnitTests/VectorUnitTests.cpp`

Testai suskirstyti i kelias grupes:

- `VectorConstructorTests`;
- `VectorCapacityTests`;
- `VectorElementAccessTests`;
- `VectorRuleOfFiveTests`;
- `VectorModifierTests`;
- `VectorIteratorAlgorithmTests`;
- `VectorComparisonTests`;
- `VectorStdVectorComparisonTests`.

Testais patikrinta:

- `Vector` konstruktoriai;
- `Vector` kopijavimas;
- `Vector` perkeltas;
- `push_back()`;

- `pop_back()`;
- `clear()`;
- `reserve()`;
- `resize()`;
- `shrink_to_fit()`;
- `insert()`;
- `erase()`;
- `assign()`;
- `swap()`;
- palyginimo operatoriai;
- iteratoriai;
- veikimas su `std::sort()`;
- veikimas su `Studentas` objektais;
- keliu `std::vector` ir `Vector` funkciju rezultatu palyginimas.

Visi unit testai praejo sėkmingai.

Unit testu rezultatu ekrano nuotrauka:

Testai paleidžiami per Visual Studio:

1. Atidaryti projektą su Visual Studio.
2. Virsutiniame meniu pasirinkti Test.
3. Atidaryti Test Explorer.
4. Paspausti Run All Tests.

1.13 Testavimo failai

Installerio testavimui pateikiami failai:

```
test_files/studentai10000.txt
test_files/studentai100000.txt
```

1.14 Doxygen dokumentacija

Projektas dokumentuotas naudojant **Doxygen**.

Dokumentacijoje aprašytos klasės:

- `Zmogus`;
- `Studentas`;
- `Vector<T>`.

Sugeneruota dokumentacija pateikiama:

Dokumentacijos tipas	Vieta projekte
HTML	docs/html/index.html

Dokumentacijos tipas	Vieta projekte
LaTeX	docs/↔ latex/
PDF	docs/refman.pdf

1.15 Diegimo failas

Programos diegimui parengtas Inno Setup scenarijus:

installer/setup.iss

Diegimo failas idiegia programa i:

C:/Program Files/VU/Ignas-Simaitis

Diegimo failo kurimas:

1. Sukompiliuoti programa Release režimu.
2. Atidaryti installer/setup.iss su Inno Setup.
3. Paspausti Compile.
4. Sugeneruojamas Setup.exe.

1.16 Versiju aprasas

1.16.1 v3.0

Sukurta nuosava `Vector<T>` klase ir pritaikyta studentu programai.

Pagrindiniai pakeitimai:

- sukurta sablonine `Vector<T>` klase;
- realizuota dauguma pagrindiniu `std::vector` funkciju;
- `Studentas` namu darbu pazymiams naudoja `Vector<int>`;
- sukurta programos versija su `Vector<Studentas>`;
- atlikti `std::vector` ir `Vector` spartos tyrimai;
- suskaiciuoti atminties perskirstymai;
- prideti `Vector` unit testai;
- `Vector.h` itrauktas i Doxygen dokumentacija;
- parengtas Inno Setup diegimo scenarijus.

1.16.2 v2.0

Sioje versijoje projektas papildytas unit testais ir Doxygen dokumentacija.

Pagrindiniai pakeitimai:

- prideti unit testai `Studentas` klasei;
- patikrinti Rule of Five metodai;
- patikrinti ivesties ir isvesties operatoriai `>>` ir `<<`;
- sugeneruota Doxygen HTML dokumentacija;
- sugeneruota Doxygen LaTeX dokumentacija;

- parengtas dokumentacijos PDF failas;
 - atnaujintas `README.md` failas;
 - patikrintas projekto kompiliavimas su CMake.
-

1.16.3 v1.5

Sioje versijoje programa papildyta paveldejimu.

Buvo sukurta abstrakti bazine klase `Zmogus`, is kurios paveldi `Studentas` klase.

Programa islaiko v1.2 versijos logika: veikia Rule of Five, ivesties ir isvesties operatoriai bei ankstesni testai.

1.16.4 v1.2

Sioje versijoje `Studentas` klase papildyta Rule of Five realizacija.

Prideta:

- destruktorius;
- kopijavimo konstruktorius;
- kopijavimo priskyrimo operatorius;
- perkavimo konstruktorius;
- perkavimo priskyrimo operatorius.

Taip pat realizuoti ivesties ir isvesties operatoriai `>>` ir `<<`.

1.16.5 v1.1

Sioje versijoje `Studentas` struktura pakeista i klase.

Pagrindiniai pakeitimai:

- `Studentas` struktura pakeista i klase;
 - studentu duomenys perkelti i privacius laukus;
 - prideti getteriai ir setteriai;
 - realizuoti konstruktoriai ir destruktorius;
 - atnaujintos funkcijos, kurios dirba su `Studentas` objektais;
 - atliktas `struct` ir `class` versiju palyginimas;
 - atlikta analize su `O1`, `O2` ir `O3` optimizavimo flag'ais.
-

1.16.6 v1.0

Sioje versijoje pridetas darbas su trimis konteineriais:

- `std::vector`;
- `std::list`;
- `std::deque`.

Taip pat buvo pridetas studentu skirstymas i dvi grupes, atliktas konteineriu tyrimas ir paruostas `CMakeLists.txt`.

1.16.7 v0.4

Sioje versijoje pridetas studentu failu generavimas, studentu skirstymas i dvi grupes ir pradinis veikimo spartos tyrimas su `std::vector`.

1.16.8 v0.3

Sioje versijoje patobulinta ivesties validacija, pridetas klaidu tikrinimas ir kodas isskaidytas i .h ir .cpp failus.

1.16.9 v0.2

Sioje versijoje pridetas duomenu nuskaitymas is failo ir studentu rusiavimas.

1.16.10 v0.1

Sioje versijoje realizuotas rankinis studentu duomenu ivedimas ir galutinio balo skaiciavimas pagal vidurki arba mediana.

1.16.11 v.pradine

Sukurta pradine studento duomenu struktura ir realizuotas pradinis vidurkio bei medianos skaiciavimas.

Chapter 2

Directory Hierarchy

2.1 Directories

Objektinis	19
Studentas.h	43
Vector.h	44
Zmogus.h	52

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Vector< T >	27
Zmogus	38
Studentas	21

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	Klase skirta saugoti studento duomenis	21
Vector< T >	Paprastas dinaminio masyvo konteineris, panasus i std::vector	27
Zmogus	Abstrakti bazine klase, skirta saugoti bendrus duomenis apie zmogu	38

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

Objektinis/ Studentas.h	43
Objektinis/ Vector.h	44
Objektinis/ Zmogus.h	52

Chapter 6

Directory Documentation

6.1 Objektinis Directory Reference

Files

- file [Studentas.h](#)
- file [Vector.h](#)
- file [Zmogus.h](#)

Chapter 7

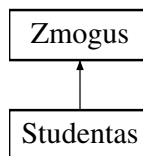
Class Documentation

7.1 Studentas Class Reference

Klase skirta saugoti studento duomenis.

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Tuscias konstruktorius, kuris sukuria [Studentas](#) objekta su numatytomis reikšmemis.
- [Studentas](#) (const std::string &vardas, const std::string &pavarde)
Konstruktorius su studento vardu ir pavarde.
- [Studentas](#) (const std::string &vardas, const std::string &pavarde, const [Vector](#)< int > &pazymiai, int egzaminas)
Konstruktorius su studento vardu, pavarde, namu darbu pazymiais ir egzamino pazymiu.
- [~Studentas](#) ()
Destruktorius.
- [Studentas](#) (const [Studentas](#) &kitas)
Kopijavimo konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &kitas)
Kopijavimo priskyrimo operatorius.
- [Studentas](#) ([Studentas](#) &&kitas) noexcept
Perkelimo konstruktorius.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&kitas) noexcept
Perkelimo priskyrimo operatorius.
- const [Vector](#)< int > & [getPazymiai](#) () const
Grazina studento namu darbu pazymius.
- int [getEgzaminas](#) () const
Grazina studento egzamino pazymi.
- double [getRezultatas](#) () const
Grazina studento galutini rezultata.
- void [setPazymiai](#) (const [Vector](#)< int > &namuDarbai)
Nustato studento namu darbu pazymius.

- void [setEgzaminas](#) (int egzaminas)
Nustato studento egzamino pazymi.
- void [setRezultatas](#) (double rezultatas)
Nustato studento galutini rezultata.
- void [pridetiPazymi](#) (int pazymys)
Prideda nauja pazymi prie studento namu darbu pazymiu.
- void [read](#) (std::istream &in) override
Nuskaityto studento duomenis is srauto.
- void [print](#) (std::ostream &out) const override
Isveda studento duomenis i srauta.

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Konstruktorius su zmogaus vardu ir pavarde.
- virtual [~Zmogus](#) ()
Virtualus Destruktorius.
- std::string [getVardas](#) () const
Grazina zmogaus varda.
- std::string [getPavarde](#) () const
Grazina zmogaus pavarde.
- void [setVardas](#) (const std::string &vardas)
Nustato zmogaus varda.
- void [setPavarde](#) (const std::string &pavarde)
Nustato zmogaus pavarde.

Static Public Member Functions

- static int [getSunaikintuObjektuKiekis](#) ()
Grazina sunaikintu [Studentas](#) objektu kieki.

Friends

- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &studentas)
Ivesties operatorius, kuris leidzia nuskaityti studento duomenis is srauto.
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &studentas)
Isvesties operatorius, kuris leidzia isvesti studento duomenis i srauta.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas_](#)
Zmogaus vardas.
- std::string [pavarde_](#)
Zmogaus pavarde.

7.1.1 Detailed Description

Klase skirta saugoti studento duomenis.

[Studentas](#) klase paveldi is [Zmogus](#) klases. Ji saugo studento namu darbu pazymius, egzamino pazymi ir galutini rezultata.

Klase turi Rule of Five metodus, kad objektus butu galima teisingai kopijuoti, priskirti ir perkelti.

Taip pat yra statinis kintamasis, kuris seka kiek [Studentas](#) klases objektu buvo sunaikinta, Kad butu galima atlikti dekonstruktoriaus testavima ir stebeti atminties valymus.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 Studentas() [1/5]

```
Studentas::Studentas ()
```

Tuscias konstruktorius, kuris sukuria [Studentas](#) objekta su numatytomis reikšmemis.

7.1.2.2 Studentas() [2/5]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde)
```

Konstruktorius su studento vardu ir pavarde.

Parameters

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavarde.

7.1.2.3 Studentas() [3/5]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const Vector< int > & pazymiai,
    int egzaminas)
```

Konstruktorius su studento vardu, pavarde, namu darbu pazymiais ir egzamino pazymiu.

Parameters

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavarde.
<i>pazymiai</i>	Studento namu darbu pazymiai.
<i>egzaminas</i>	Studento egzamino pazymys.

7.1.2.4 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Destruktorius.

7.1.2.5 Studentas() [4/5]

```
Studentas::Studentas (
    const Studentas & kitas)
```

Kopijavimo konstruktorius.

Parameters

<i>kitas</i>	Kitas Studentas objektas, kurio duomenys bus kopijuojami.
--------------	---

7.1.2.6 Studentas() [5/5]

```
Studentas::Studentas (
    Studentas && kitas) [noexcept]
```

Perkélimo konstruktorius.

Parameters

<i>kitas</i>	Kitas Studentas objektas, kurio duomenys bus perkelti.
--------------	--

7.1.3 Member Function Documentation**7.1.3.1 getEgzaminas()**

```
int Studentas::getEgzaminas () const
```

Grazina studento egzamino pazymi.

Returns

Studento egzamino pazymys.

7.1.3.2 getPazymiai()

```
const Vector< int > & Studentas::getPazymiai () const
```

Grazina studento namu darbu pazymius.

Returns

Studento namu darbu pazymiai.

7.1.3.3 getRezultatas()

```
double Studentas::getRezultatas () const
```

Grazina studento galutini rezultata.

Returns

Studento galutinis rezultatas.

7.1.3.4 getSunaikintuObjektuKiekis()

```
int Studentas::getSunaikintuObjektuKiekis () [static]
```

Grazina sunaikintu [Studentas](#) objektu kiekis.

Returns

Sunaikintu [Studentas](#) objektu kiekis.

7.1.3.5 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas)
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>kitas</i>	Kitas Studentas objektas, kurio duomenys bus kopijuojami.
--------------	---

Returns

Nuoroda i esama [Studentas](#) objektą.

7.1.3.6 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && kitas) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Parameters

<i>kitas</i>	Kitas Studentas objektas, kurio duomenys bus perkelti.
--------------	--

Returns

Nuoroda i esama [Studentas](#) objektą.

7.1.3.7 pridetiPazymi()

```
void Studentas::pridetiPazymi (  
    int pazymys)
```

Prideda nauja pazymi prie studento namu darbu pazymiu.

Parameters

<i>pazymys</i>	Naujas pazymys.
----------------	-----------------

7.1.3.8 print()

```
void Studentas::print (  
    std::ostream & out) const [override], [virtual]
```

Isveda studento duomenis i srauta.

Parameters

<i>out</i>	Isvesties srautas, i kuri bus isvedami studento duomenys.
------------	---

Implements [Zmogus](#).

7.1.3.9 read()

```
void Studentas::read (  
    std::istream & in) [override], [virtual]
```

Nuskaito studento duomenis is srauto.

Parameters

<i>in</i>	Ivesties srautas, is kurio bus nuskaitomi studento duomenys.
-----------	--

Implements [Zmogus](#).

7.1.3.10 setEgzaminas()

```
void Studentas::setEgzaminas (
    int egzaminas)
```

Nustato studento egzamino pazymi.

Parameters

<i>egzaminas</i>	Studento egzamino pazymys.
------------------	----------------------------

7.1.3.11 setPazymiai()

```
void Studentas::setPazymiai (
    const Vector< int > & namuDarbai)
```

Nustato studento namu darbu pazymius.

Parameters

<i>namuDarbai</i>	Studento namu darbu pazymiai.
-------------------	-------------------------------

7.1.3.12 setRezultatas()

```
void Studentas::setRezultatas (
    double rezultatas)
```

Nustato studento galutini rezultata.

Parameters

<i>rezultatas</i>	Studento galutinis rezultatas.
-------------------	--------------------------------

7.1.4 Friends And Related Symbol Documentation**7.1.4.1 operator<<**

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & studentas) [friend]
```

Isvesties operatorius, kuris leidzia isvesti studento duomenis i srauta.

Parameters

<i>out</i>	Isvesties srautas, i kuri bus isvedami studento duomenys.
<i>studentas</i>	Studentas objektas, kurio duomenys bus isvedami.

Returns

Nuoroda i isvesties srauta.

7.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & studentas) [friend]
```

Ivesties operatorius, kuris leidzia nuskaityti studento duomenis is srauto.

Parameters

<i>in</i>	Ivesties srautas, is kurio bus nuskaitomi studento duomenys.
<i>studentas</i>	Studentas objektas, i kuri bus saugomi nuskaitomi duomenys.

Returns

Nuoroda i ivesties srauta.

The documentation for this class was generated from the following file:

- Objektinis/[Studentas.h](#)

7.2 Vector< T > Class Template Reference

Paprastas dinaminio masyvo konteineris, panasus i std::vector.

```
#include <Vector.h>
```

Public Member Functions

- [Vector](#) ()
Sukuria tuscia [Vector](#) konteineri.
- [Vector](#) (std::size_t kiekis)
Sukuria [Vector](#) konteineri su nurodytu elementu kiekiu.
- [Vector](#) (std::size_t kiekis, const T &reiksme)
Sukuria [Vector](#) konteineri su nurodytu elementu kiekiu ir pradiniu elementu reiksme.
- [Vector](#) (std::initializer_list< T > sarasas)
Sukuria [Vector](#) konteineri is inicializavimo saraso.
- [~Vector](#) ()
Atlaisvina konteinerio naudojama atminti.
- std::size_t [size](#) () const
Grazina elementu kieki.
- std::size_t [max_size](#) () const
Grazina teorini didziausia elementu kieki.
- std::size_t [capacity](#) () const
Grazina rezervuotos vietos kieki.
- bool [empty](#) () const
Patikrina, ar konteineris tuscias.
- T * [data](#) ()
Grazina rodykle i saugomu elementu masyva.
- const T * [data](#) () const
Grazina rodykle i saugomu elementu masyva.
- T * [begin](#) ()
Grazina iteratoriu i pirma elementa.
- const T * [begin](#) () const
Grazina const iteratoriu i pirma elementa.
- T * [end](#) ()
Grazina iteratoriu uz paskutinio elemento.
- const T * [end](#) () const
Grazina konstantini iteratoriu uz paskutinio elemento.
- const T * [cbegin](#) () const
Grazina const iteratoriu i pirma elementa.
- const T * [cend](#) () const

- Grazina const iteratoriu uz paskutinio elemento.*
- T & [operator\[\]](#) (std::size_t index)
Grazina elementa pagal indeksa be ribu tikrinimo.
- const T & [operator\[\]](#) (std::size_t index) const
Grazina elementa pagal indeksa be ribu tikrinimo.
- T & [at](#) (std::size_t index)
Grazina elementa pagal indeksa su ribu tikrinimu.
- const T & [at](#) (std::size_t index) const
Grazina elementa pagal indeksa su ribu tikrinimu.
- T & [front](#) ()
Grazina pirma elementa.
- const T & [front](#) () const
Grazina pirma elementa.
- T & [back](#) ()
Grazina paskutini elementa.
- const T & [back](#) () const
Grazina paskutini elementa.
- void [reserve](#) (std::size_t naujaTalpa)
Rezervuoja vietas nurodytam elementu kiekiui.
- void [resize](#) (std::size_t naujasDydis)
Pakeicia konteinerio dydi.
- void [resize](#) (std::size_t naujasDydis, const T &reiksme)
Pakeicia konteinerio dydi ir naujiems elementams priskiria reiksme.
- void [shrink_to_fit](#) ()
Sumazina talpa iki dabartinio elementu kiekio.
- void [push_back](#) (const T &reiksme)
Prideda elementa i konteinerio pabaiga.
- void [push_back](#) (T &&reiksme)
Prideda perkelta elementa i konteinerio pabaiga.
- template<typename... Args>
T & [emplace_back](#) (Args &&... args)
Sukuria nauja elementa konteinerio pabaigoje.
- template<typename... Args>
T * [emplace](#) (T *pozicija, Args &&... args)
Sukuria nauja elementa nurodytoje pozicijoje.
- void [pop_back](#) ()
pasalina paskutini elementa.
- T * [insert](#) (T *pozicija, const T &reiksme)
Iterpia viena elementa pries nurodyta pozicija.
- T * [insert](#) (T *pozicija, std::size_t kiekis, const T &reiksme)
Iterpia kelias vienodas reikses pries nurodyta pozicija.
- T * [insert](#) (T *pozicija, std::initializer_list< T > sarasas)
Iterpia inicializavimo saraso elementus pries nurodyta pozicija.
- T * [erase](#) (T *pozicija)
Pasalina elementa is nurodytos pozicijos.
- T * [erase](#) (T *pirma, T *paskutine)
Pasalina elementus is nurodytos pozicijos intervalo.
- void [swap](#) ([Vector](#) &kitas) noexcept
Apkeicia du [Vector](#) konteinerio duomenis.
- bool [operator==](#) (const [Vector](#) &kitas) const
Palygina du [Vector](#) objektus.

- bool `operator!=` (const `Vector` &kitas) const
Palygina du `Vector` objektus.
- void `assign` (std::size_t kiekis, const T &reiksme)
Pakeicia `Vector` turinti nurodytu kiekiu vienodu elementu.
- void `assign` (std::initializer_list< T > sarasas)
Pakeicia `Vector` turini inicializavimo sarasu.
- void `clear` ()
Pasalina visus elementus, bet palieka rezervuota talpa.
- `Vector` (const `Vector` &kitas)
Sukuria `Vector` kopija is kito `Vector` objekto.
- `Vector` & `operator=` (const `Vector` &kitas)
Priskiria kito `Vector` objekto kopija.
- `Vector` (`Vector` &&kitas) noexcept
Sukuria `Vector` perimant kito `Vector` objekto duomenis.
- `Vector` & `operator=` (`Vector` &&kitas) noexcept
Priskiria `Vector` turini is inicializavimo saraso.
- `Vector` & `operator=` (std::initializer_list< T > sarasas)
Priskiria `Vector` turini is inicializavimo saraso.

7.2.1 Detailed Description

`template<typename T>`

`class Vector< T >`

Paprastas dinaminio masyvo konteineris, panasus i std::vector.

`Vector` klase realizuota kaip sablonine klase, todėl gali saugoti skirtingu tipu elementus. Viduje naudojamas dinaminis masyvas, dabartinis elementu kiekis ir rezervuota talpa. Kai elementu kiekis pasiekia talpa, konteineris rezervuoja didesni masyva ir perkelia senus elementus.

Template Parameters

<code>T</code>	Saugomu elementu tipas.
----------------	-------------------------

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Vector() [1/6]

`template<typename T>`

`Vector< T >::Vector () [inline]`

Sukuria tuscia `Vector` konteineri.

7.2.2.2 Vector() [2/6]

`template<typename T>`

`Vector< T >::Vector (`
 std::size_t kiekis) [inline]

Sukuria `Vector` konteineri su nurodytu elementu kiekiu.

Parameters

<code>kiekis</code>	Pradinis elementu kiekis.
---------------------	---------------------------

7.2.2.3 Vector() [3/6]

```
template<typename T>
Vector< T >::Vector (
    std::size_t kiekis,
    const T & reiksme) [inline]
```

Sukuria **Vector** konteineri su nurodytu elementu kiekiu ir pradiniu elementu reiksme.

Parameters

<i>kiekis</i>	Pradinis elementu kiekis.
<i>reiksme</i>	Pradine elementu reiksme.

7.2.2.4 Vector() [4/6]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > sarasas) [inline]
```

Sukuria **Vector** konteineri is inicializavimo saraso.

Parameters

<i>sarasas</i>	Pradines elementu reiksmes.
----------------	-----------------------------

7.2.2.5 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

Atlaisvina konteinerio naudojama atminti.

7.2.2.6 Vector() [5/6]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & kitas) [inline]
```

Sukuria **Vector** kopija is kito **Vector** objekto.

Parameters

<i>kitas</i>	Kopijuojamas Vector objektas.
--------------	--------------------------------------

7.2.2.7 Vector() [6/6]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && kitas) [inline], [noexcept]
```

Sukuria **Vector** perimant kito **Vector** objekto duomenis.

Parameters

<i>kitas</i>	Vector objektas, is kurio duomenys perkeliama.
--------------	---

7.2.3 Member Function Documentation

7.2.3.1 assign() [1/2]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > sarasas) [inline]
```

Pakeicia [Vector](#) turini inicializavimo *sarasas*.

Parameters

<i>sarasas</i>	Naujos elementu reikšmės.
----------------	---------------------------

7.2.3.2 assign() [2/2]

```
template<typename T>
void Vector< T >::assign (
    std::size_t kiekis,
    const T & reiksme) [inline]
```

Pakeicia [Vector](#) turinti nurodytu kiekiu vienodu elementu.

Parameters

<i>kiekis</i>	Naujai priskiriamų elementų kiekis.
<i>reiksme</i>	Reikšmė, kuria bus priskirti nauji elementai.

7.2.3.3 at() [1/2]

```
template<typename T>
T & Vector< T >::at (
    std::size_t index) [inline]
```

Grazina elementa pagal indeksą su ribų tikrinimu.

7.2.3.4 at() [2/2]

```
template<typename T>
const T & Vector< T >::at (
    std::size_t index) const [inline]
```

Grazina elementą pagal indeksą su ribų tikrinimu.

7.2.3.5 back() [1/2]

```
template<typename T>
T & Vector< T >::back () [inline]
```

Grazina paskutini elementą.

7.2.3.6 back() [2/2]

```
template<typename T>
const T & Vector< T >::back () const [inline]
```

Grazina paskutini elementą.

7.2.3.7 begin() [1/2]

```
template<typename T>
T * Vector< T >::begin () [inline]
```

Grazina iteratorių į pirmą elementą.

7.2.3.8 begin() [2/2]

```
template<typename T>
const T * Vector< T >::begin () const [inline]
```

Grazina const iteratoriu i pirma elementa.

7.2.3.9 capacity()

```
template<typename T>
std::size_t Vector< T >::capacity () const [inline]
```

Grazina rezervuotos vietos kiekis.

7.2.3.10 cbegin()

```
template<typename T>
const T * Vector< T >::cbegin () const [inline]
```

Grazina const iteratoriu i pirma elementa.

7.2.3.11 cend()

```
template<typename T>
const T * Vector< T >::cend () const [inline]
```

Grazina const iteratoriu uz paskutinio elemento.

7.2.3.12 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

Pasalina visus elementus, bet palieka rezervuota talpa.

7.2.3.13 data() [1/2]

```
template<typename T>
T * Vector< T >::data () [inline]
```

Grazina rodykle i saugomu elementu masyva.

7.2.3.14 data() [2/2]

```
template<typename T>
const T * Vector< T >::data () const [inline]
```

Grazina rodykle i saugomu elementu masyva.

7.2.3.15 emplace()

```
template<typename T>
template<typename... Args>
T * Vector< T >::emplace (
    T * pozicija,
    Args &&... args) [inline]
```

Sukuria nauja elementa nurodytoje pozicijoje.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentu tipai
-------------	---------------------------------

Parameters

<i>pozicija</i>	Iteratorius i vieta, kur bus iterpiamas elementas.
<i>args</i>	Argumentai, perduodami elemento konstruktoriui.

Returns

Iteratorius i naujai sukurta elementa.

7.2.3.16 `emplace_back()`

```
template<typename T>
template<typename... Args>
T & Vector< T >::emplace_back (
    Args &&... args) [inline]
```

Sukuria nauja elementa konteinerio pabaigoje;.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentu tipai.
-------------	----------------------------------

Parameters

<i>args</i>	Argumentai, perduodami elemento konstruktoriui.
-------------	---

Returns

Nuoroda i naujai sukurta elementa.

7.2.3.17 `empty()`

```
template<typename T>
bool Vector< T >::empty () const [inline]
```

Patikrina, ar konteineris tuscias.

7.2.3.18 `end()` [1/2]

```
template<typename T>
T * Vector< T >::end () [inline]
```

Grazina iteratoriu uz paskutinio elemento.

7.2.3.19 `end()` [2/2]

```
template<typename T>
const T * Vector< T >::end () const [inline]
```

Grazina konstantini iteratoriu uz paskutinio elemento.

7.2.3.20 `erase()` [1/2]

```
template<typename T>
T * Vector< T >::erase (
    T * pirma,
    T * paskutine) [inline]
```

Pasalina elementus is nurodytos pozicijos intervalo.

Parameters

<i>pirma</i>	Iteratorius i pirma salinama elementa
<i>paskutine</i>	Iteratorius uz paskutinio salinamo elemento

Returns

Iteratorius i vieta, kurioje prasideda pasalintas intervalas

7.2.3.21 erase() [2/2]

```
template<typename T>
T * Vector< T >::erase (
    T * pozicija) [inline]
```

Pasalina elementa is nurodytos pozicijos.

Parameters

<i>pozicija</i>	Iteratoriaus i salinama elementa
-----------------	----------------------------------

Returns

Iteratorius i elementa po pasalinto elemento

7.2.3.22 front() [1/2]

```
template<typename T>
T & Vector< T >::front () [inline]
```

Grazina pirma elementa.

7.2.3.23 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const [inline]
```

Grazina pirma elementa.

7.2.3.24 insert() [1/3]

```
template<typename T>
T * Vector< T >::insert (
    T * pozicija,
    const T & reiksme) [inline]
```

Iterpia viena elementa pries nurodyta pozicija.

Parameters

<i>pozicija</i>	Iteratorius i vieta, pries kuria iterpiama.
<i>reiksme</i>	Iterpiama reiksme.

Returns

Iteratorius i iterpta elementa.

7.2.3.25 insert() [2/3]

```
template<typename T>
T * Vector< T >::insert (
    T * pozicija,
    std::initializer_list< T > sarasas) [inline]
```

Iterpia inicializavimo saraso elementus prieš nurodyta pozicija.

Parameters

<i>pozicija</i>	Iteratorius i vieta, prieš kuria iterpiama
<i>sarasas</i>	Iterpiamos reiksmes

Returns

Iteratorius i pirma iterpta elementa

7.2.3.26 insert() [3/3]

```
template<typename T>
T * Vector< T >::insert (
    T * pozicija,
    std::size_t kiekis,
    const T & reiksme) [inline]
```

Iterpia kelias vienodas reikses prieš nurodyta pozicija.

Parameters

<i>pozicija</i>	Iteratorius i vieta, prieš kuria iterpiama.
<i>kiekis</i>	Kiek elementu iterpti
<i>reiksme</i>	Iterpiama reiksme

Returns

Iteratorius i pirma iterpta elementa.

7.2.3.27 max_size()

```
template<typename T>
std::size_t Vector< T >::max_size () const [inline]
```

Grazina teorini didziausia elementu kiekis.

Returns

Didziausias galimas elementu kiekis.

7.2.3.28 operator"!="()

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & kitas) const [inline]
```

Palygina du Vector objektus.

Parameters

<i>kitas</i>	Kitas Vector objektas, su kuriuo bus lyginamas esamas objektas.
--------------	---

Returns

true jeigu dydis arba bent vienas elementas nesutampa.

7.2.3.29 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & kitas) [inline]
```

Priskiria kito [Vector](#) objekto kopiją.

Parameters

<i>kitas</i>	Kopijuojamas Vector objektas.
--------------	---

Returns

Nuoroda i si [Vector](#) objekta.

7.2.3.30 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > sarasas) [inline]
```

Priskiria [Vector](#) turini is inicializavimo saraso.

Parameters

<i>sarasas</i>	Naujos elementu reikšmės
----------------	--------------------------

Returns

nuoroda i si [Vector](#) objekta,

7.2.3.31 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && kitas) [inline], [noexcept]
```

Priskiria [Vector](#) turini is inicializavimo saraso.

Parameters

<i>sarasas</i>	Naujos elementu reikšmės.
----------------	---------------------------

Returns

Nuoroda i si [Vector](#) objekta.

7.2.3.32 operator==()

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & kitas) const [inline]
```

Palygina du [Vector](#) objektus.

Parameters

<i>kitas</i>	Kitas Vector objektas, su kuriuo bus lyginamas esamas objektas
--------------	--

Returns

true jeigu dydis ir visi elementai sutampa.

7.2.3.33 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    std::size_t index) [inline]
```

Grazina elementa pagal indeksa be ribu tikrinimo.

7.2.3.34 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    std::size_t index) const [inline]
```

Grazina elementa pagal indeksa be ribu tikrinimo.

7.2.3.35 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

pasalina paskutini elementa.

7.2.3.36 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & reiksme) [inline]
```

Prideda elementa i konteinerio pabaiga.

7.2.3.37 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && reiksme) [inline]
```

Prideda perkelta elementa i konteinerio pabaiga.

Parameters

<i>reiksme</i>	Perkeliamas reiksme
----------------	---------------------

7.2.3.38 reserve()

```
template<typename T>
void Vector< T >::reserve (
    std::size_t naujaTalpa) [inline]
```

Rezervuoja vietas nurodytam elementu kiekiui.
Jei nauja talpa yra mazesne arba lygi dabartinei, nieko nedaro.

7.2.3.39 `resize()` [1/2]

```
template<typename T>
void Vector< T >::resize (
    std::size_t naujasDydis) [inline]
```

Pakeicia konteinerio dydi.

Jei naujas dydis yra didesnis uz dabartini, nauji elementai inicializuojami numatytaja reiksme.

Parameters

<i>naujasDydis</i>	Naujas konteinerio dydis.
--------------------	---------------------------

7.2.3.40 `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    std::size_t naujasDydis,
    const T & reiksme) [inline]
```

Pakeicia konteinerio dydi ir naujiems elementams priskiria reiksme.

Parameters

<i>naujasDydis</i>	Naujas konteinerio dydis.
<i>reiksme</i>	Reiksme, kuria uzpildomi nauji elementai.

7.2.3.41 `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

Sumazina talpa iki dabartinio elementu kiekio.

7.2.3.42 `size()`

```
template<typename T>
std::size_t Vector< T >::size () const [inline]
```

Grazina elementu kiekį.

7.2.3.43 `swap()`

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & kitas) [inline], [noexcept]
```

Apkeicia du `Vector` konteinerio duomenis.

Parameters

<i>kitas</i>	Kitas <code>Vector</code> konteineris, su kuriuo kečiami duomenys.
--------------	--

The documentation for this class was generated from the following file:

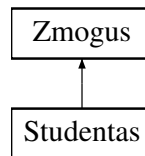
- Objektinis/[Vector.h](#)

7.3 Zmogus Class Reference

Abstrakti bazine klase, skirta saugoti bendrus duomenis apie zmogu.

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Konstruktorius su zmogaus vardu ir pavarde.
- virtual [~Zmogus](#) ()
Virtualus Destruktorius.
- std::string [getVardas](#) () const
Grazina zmogaus varda.
- std::string [getPavarde](#) () const
Grazina zmogaus pavarde.
- void [setVardas](#) (const std::string &vardas)
Nustato zmogaus varda.
- void [setPavarde](#) (const std::string &pavarde)
Nustato zmogaus pavarde.
- virtual void [read](#) (std::istream &in)=0
Virtuali funkcija, skirta skaityti zmogaus duomenis is srauto.
- virtual void [print](#) (std::ostream &out) const =0
Virtuali funkcija, skirta spausdinti zmogaus duomenis i srauta.

Protected Attributes

- std::string [vardas_](#)
Zmogaus vardas.
- std::string [pavarde_](#)
Zmogaus pavarde.

7.3.1 Detailed Description

Abstrakti bazine klase, skirta saugoti bendrus duomenis apie zmogu.

Sioje klaseje saugomas vardas ir pavarde. Klase naudojama kaip bazine klase [Studentas](#) klasei.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus ()
```

Numatytasis konstruktorius.

7.3.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde)
```

Konstruktorius su zmogaus vardu ir pavarde.

Parameters

<i>vardas</i>	Zmogaus vardas.
<i>pavarde</i>	Zmogaus pavarde.

7.3.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [virtual]
```

Virtualus Destruktorius.

7.3.3 Member Function Documentation

7.3.3.1 getPavarde()

```
std::string Zmogus::getPavarde () const
```

Grazina zmogaus pavarde.

Returns

Zmogaus pavarde.

7.3.3.2 getVardas()

```
std::string Zmogus::getVardas () const
```

Grazina zmogaus varda.

Returns

Zmogaus vardas.

7.3.3.3 print()

```
virtual void Zmogus::print (
    std::ostream & out) const [pure virtual]
```

Virtuali funkcija, skirta spausdinti zmogaus duomenis i srauta.

Parameters

<i>out</i>	Isvesties srautas, i kuri bus spausdinami duomenys.
------------	---

Implemented in [Studentas](#).

7.3.3.4 read()

```
virtual void Zmogus::read (
    std::istream & in) [pure virtual]
```

Virtuali funkcija, skirta skaityti zmogaus duomenis is srauto.

Parameters

<i>in</i>	Ivesties srautas, is kurio bus skaitomi duomenys.
-----------	---

Implemented in [Studentas](#).

7.3.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & pavarde)
```

Nustato zmogaus pavarde.

Parameters

<i>pavarde</i>	Nauja pavarde.
----------------	----------------

7.3.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & vardas)
```

Nustato zmogaus vardą.

Parameters

<i>vardas</i>	Naujas vardas.
---------------	----------------

7.3.4 Member Data Documentation

7.3.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Zmogaus pavarde.

7.3.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Zmogaus vardas.

The documentation for this class was generated from the following file:

- Objektinis/[Zmogus.h](#)

Chapter 8

File Documentation

8.1 Objektinis/Studentas.h File Reference

```
#include "Zmogus.h"
#include "Vector.h"
#include <string>
#include <vector>
#include <iostream>
```

Classes

- class [Studentas](#)
Klase skirta saugoti studento duomenis.

Macros

- #define [STUDENTAS_H](#)

8.1.1 Macro Definition Documentation

8.1.1.1 STUDENTAS_H

```
#define STUDENTAS_H
```

8.2 Studentas.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #ifndef STUDENTAS_H
00003 #define STUDENTAS_H
00004 #include "Zmogus.h"
00005 #include "Vector.h"
00006
00007 #include <string>
00008 #include <vector>
00009 #include <iostream>
00010
00026 class Studentas : public Zmogus
00027 {
00028 private:
00032     Vector<int> pazymiai_;
00036     int egzaminas_;
00040     double rezultatas_;
00044     static int sunaikintuObjektuKiekis_;
00045
00046 public:
00050     Studentas();
00051
00057     Studentas(const std::string& vardas, const std::string& pavarde);
```

```

00058
00066     Studentas(const std::string& vardas, const std::string& pavarde, const Vector<int>& pazymiai, int
egzaminas);
00067
00072     ~Studentas()
00073     {
00074         pazymiai_.clear();
00075         egzaminas_ = 0;
00076         rezultatas_ = 0.0;
00077         sunaikintuObjektuKiekis_++;
00078     }
00079
00080
00085     static int getSunaikintuObjektuKiekis();
00086
00091     Studentas(const Studentas& kitas);
00092
00098     Studentas& operator=(const Studentas& kitas);
00099
00104     Studentas(Studentas&& kitas) noexcept;
00105
00111     Studentas& operator=(Studentas&& kitas) noexcept;
00112
00117     const Vector<int>& getPazymiai() const;
00118
00123     int getEgzaminas() const;
00124
00129     double getRezultatas() const;
00130
00135     void setPazymiai(const Vector<int>& namuDarbai);
00136
00141     void setEgzaminas(int egzaminas);
00142
00147     void setRezultatas(double rezultatas);
00148
00153     void pridetiPazymi(int pazymys);
00154
00159     void read(std::istream& in) override;
00160
00165     void print(std::ostream& out) const override;
00166
00173     friend std::istream& operator>(std::istream& in, Studentas& studentas);
00174
00181     friend std::ostream& operator<(std::ostream& out, const Studentas& studentas);
00182
00183 };
00184
00185 #endif // STUDENTAS_H

```

8.3 Objektinis/Vector.h File Reference

```

#include <cstdint>
#include <stdexcept>
#include <initializer_list>
#include <utility>
#include <limits>

```

Classes

- class [Vector< T >](#)
Paprastas dinaminio masyvo konteineris, panasus i std::vector.

Macros

- #define [VECTOR_H](#)

8.3.1 Macro Definition Documentation

8.3.1.1 VECTOR_H

```
#define VECTOR_H
```

8.4 Vector.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #ifndef VECTOR_H
00003 #define VECTOR_H
00004
00005 #include <cstdlib>
00006 #include <stdexcept>
00007 #include <initializer_list>
00008 #include <utility>
00009 #include <limits>
00010 template <typename T>
00011 class Vector
00012 {
00013 private:
00014     T* duomenys_;
00015
00016     std::size_t dydis_;
00017
00018     std::size_t talpa_;
00019
00020 public:
00021     Vector()
00022         : duomenys_(nullptr), dydis_(0), talpa_(0)
00023     {}
00024
00025     Vector(std::size_t kiekis)
00026         : duomenys_(nullptr), dydis_(kiekis), talpa_(kiekis)
00027     {
00028         if (talpa_ > 0)
00029         {
00030             duomenys_ = new T[talpa_];
00031         }
00032     }
00033
00034     Vector(std::size_t kiekis, const T& reiksme)
00035         : duomenys_(nullptr), dydis_(kiekis), talpa_(kiekis)
00036     {
00037         if (talpa_ > 0)
00038         {
00039             duomenys_ = new T[talpa_];
00040
00041             for (std::size_t i = 0; i < dydis_; ++i)
00042             {
00043                 duomenys_[i] = reiksme;
00044             }
00045         }
00046     }
00047
00048     Vector(std::initializer_list<T> sarasas)
00049         : duomenys_(nullptr), dydis_(sarasas.size()), talpa_(sarasas.size())
00050     {
00051         if (talpa_ > 0)
00052         {
00053             duomenys_ = new T[talpa_];
00054
00055             std::size_t i = 0;
00056
00057             for (const T& item : sarasas)
00058             {
00059                 duomenys_[i] = item;
00060                 ++i;
00061             }
00062         }
00063     }
00064
00065     ~Vector()
00066     {
00067         delete[] duomenys_;
00068     }
00069
00070     std::size_t size() const
00071     {
00072         return dydis_;
00073     }
00074
00075     std::size_t max_size() const
00076     {
00077         return std::numeric_limits<std::size_t>::max() / sizeof(T);
00078     }
00079
00080     std::size_t capacity() const
00081     {
00082         return talpa_;
00083     }
00084
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123
00124
00125
00126
00127
00128
00129
00130
00131
00132
00133

```



```

00269         throw std::out_of_range("Vector is empty");
00270     }
00271     return duomenys_[dydis_ - 1];
00272 }
00273 const T& back() const
00274 {
00275     if (empty())
00276     {
00277         throw std::out_of_range("Vector is empty");
00278     }
00279     return duomenys_[dydis_ - 1];
00280 }
00281 void reserve(std::size_t naujaTalpa)
00282 {
00283     if (naujaTalpa > max_size())
00284     {
00285         throw std::length_error("Vector talpa per didele");
00286     }
00287     if (naujaTalpa <= talpa_)
00288     {
00289         return;
00290     }
00291     T* naujiDuomenys = new T[naujaTalpa];
00292     for (std::size_t i = 0; i < dydis_; ++i)
00293     {
00294         naujiDuomenys[i] = duomenys_[i];
00295     }
00296     delete[] duomenys_;
00297     duomenys_ = naujiDuomenys;
00298     talpa_ = naujaTalpa;
00299 }
00300 void resize(std::size_t naujasDydis)
00301 {
00302     if (naujasDydis > talpa_)
00303     {
00304         reserve(naujasDydis);
00305     }
00306     for (std::size_t i = dydis_; i < naujasDydis; ++i)
00307     {
00308         duomenys_[i] = T();
00309     }
00310     dydis_ = naujasDydis;
00311 }
00312 void resize(std::size_t naujasDydis, const T& reiksme)
00313 {
00314     if (naujasDydis > talpa_)
00315     {
00316         reserve(naujasDydis);
00317     }
00318     for (std::size_t i = dydis_; i < naujasDydis; ++i)
00319     {
00320         duomenys_[i] = reiksme;
00321     }
00322     dydis_ = naujasDydis;
00323 }
00324 void shrink_to_fit()
00325 {
00326     if (talpa_ == dydis_)
00327     {
00328         return;
00329     }
00330     if (dydis_ == 0)
00331     {
00332         delete[] duomenys_;
00333         duomenys_ = nullptr;
00334         talpa_ = 0;
00335         return;
00336     }
00337     T* naujiDuomenys = new T[dydis_];
00338     for (std::size_t i = 0; i < dydis_; ++i)
00339     {
00340         naujiDuomenys[i] = duomenys_[i];
00341     }
00342     delete[] duomenys_;
00343     duomenys_ = naujiDuomenys;
00344     talpa_ = dydis_;
00345 }

```

```

00384
00385 void push_back(const T& reiksme)
00386 {
00387     if (dydis_ >= talpa_)
00388     {
00389         reserve(talpa_ == 0 ? 1 : talpa_ * 2);
00390     }
00391     duomenys_[dydis_] = reiksme;
00392     ++dydis_;
00393 }
00394
00400 void push_back(T&& reiksme)
00401 {
00402     if (dydis_ == talpa_)
00403     {
00404         std::size_t naujaTalpa = (talpa_ == 0) ? 1 : talpa_ * 2;
00405         reserve(naujaTalpa);
00406     }
00407
00408     duomenys_[dydis_] = std::move(reiksme);
00409     ++dydis_;
00410 }
00411
00420 template <typename... Args>
00421 T& emplace_back(Args&&... args)
00422 {
00423     if (dydis_ == talpa_)
00424     {
00425         std::size_t naujaTalpa = (talpa_ == 0) ? 1 : talpa_ * 2;
00426         reserve(naujaTalpa);
00427     }
00428
00429     duomenys_[dydis_] = T(std::forward<Args>(args)...);
00430     ++dydis_;
00431
00432     return duomenys_[dydis_ - 1];
00433 }
00434
00443 template <typename... Args>
00444 T* emplace(T* pozicija, Args&&... args)
00445 {
00446     std::size_t indeksas = pozicija - duomenys_;
00447
00448     if (indeksas > dydis_)
00449     {
00450         throw std::out_of_range("Vector emplace pozicija uz ribu");
00451     }
00452     if (dydis_ == talpa_)
00453     {
00454         std::size_t naujaTalpa = (talpa_ == 0) ? 1 : talpa_ * 2;
00455         reserve(naujaTalpa);
00456     }
00457
00458     for (std::size_t i = dydis_; i > indeksas; --i)
00459     {
00460         duomenys_[i] = std::move(duomenys_[i - 1]);
00461     }
00462
00463     duomenys_[indeksas] = T(std::forward<Args>(args)...);
00464     ++dydis_;
00465
00466     return duomenys_ + indeksas;
00467 }
00468
00472 void pop_back()
00473 {
00474     if (empty())
00475     {
00476         throw std::out_of_range("Vector is empty");
00477     }
00478     --dydis_;
00479 }
00487
00488 T* insert(T* pozicija, const T& reiksme)
00489 {
00490     std::size_t indeksas = pozicija - duomenys_;
00491
00492     if (indeksas > dydis_)
00493     {
00494         throw std::out_of_range("Insert position out of range");
00495     }
00496
00497     if (dydis_ >= talpa_)
00498     {
00499         reserve(talpa_ == 0 ? 1 : talpa_ * 2);
00500     }
00501

```

```

00502         for (std::size_t i = dydis_; i > indeksas; --i)
00503         {
00504             duomenys_[i] = duomenys_[i - 1];
00505         }
00506         duomenys_[indeksas] = reiksme;
00507         ++dydis_;
00508
00509         return duomenys_ + indeksas;
00510     }
00511
00512     T* insert(T* pozicija, std::size_t kiekis, const T& reiksme)
00513     {
00514         std::size_t indeksas = pozicija - duomenys_;
00515
00516         if (indeksas > dydis_)
00517         {
00518             throw std::out_of_range("Insert position out of range");
00519         }
00520
00521         if (kiekis == 0)
00522         {
00523             return duomenys_ + indeksas;
00524         }
00525
00526         if (dydis_ + kiekis > talpa_)
00527         {
00528             std::size_t naujaTalpa = (talpa_ == 0) ? 1 : talpa_;
00529
00530             while (naujaTalpa < dydis_ + kiekis)
00531             {
00532                 naujaTalpa *= 2;
00533             }
00534
00535             reserve(naujaTalpa);
00536         }
00537
00538         for (std::size_t i = dydis_; i > indeksas; --i)
00539         {
00540             duomenys_[i + kiekis - 1] = duomenys_[i - 1];
00541         }
00542
00543         for (std::size_t i = 0; i < kiekis; i++)
00544         {
00545             duomenys_[indeksas + i] = reiksme;
00546         }
00547
00548         dydis_ += kiekis;
00549
00550         return duomenys_ + indeksas;
00551     }
00552
00553     T* insert(T* pozicija, std::initializer_list<T> sarasas)
00554     {
00555         std::size_t indeksas = pozicija - duomenys_;
00556         std::size_t kiekis = sarasas.size();
00557
00558         if (indeksas > dydis_)
00559         {
00560             throw std::out_of_range("Insert position out of range");
00561         }
00562
00563         if (kiekis == 0)
00564         {
00565             return duomenys_ + indeksas;
00566         }
00567
00568         if (dydis_ + kiekis > talpa_)
00569         {
00570             std::size_t naujaTalpa = (talpa_ == 0) ? 1 : talpa_;
00571
00572             while (naujaTalpa < dydis_ + kiekis)
00573             {
00574                 naujaTalpa *= 2;
00575             }
00576
00577             reserve(naujaTalpa);
00578         }
00579
00580         for (std::size_t i = dydis_ ; i > indeksas; --i )
00581         {
00582             duomenys_[i + kiekis - 1] = duomenys_[i - 1];
00583         }
00584
00585         std::size_t i = 0;
00586
00587         for (const T& reiksme : sarasas)
00588         {
00589             duomenys_[indeksas + i] = reiksme;
00590             ++i;
00591         }

```

```

00604     }
00605     dydis_ += kiekis;
00606
00607     return duomenys_ + indeksas;
00608 }
00609
00615 T* erase(T* pozicija)
00616 {
00617     std::size_t indeksas = pozicija - duomenys_;
00618
00619     if (indeksas >= dydis_)
00620     {
00621         throw std::out_of_range("Erase position out of range");
00622     }
00623
00624     for (std::size_t i = indeksas; i < dydis_ - 1; ++i)
00625     {
00626         duomenys_[i] = duomenys_[i + 1];
00627     }
00628
00629     --dydis_;
00630     return duomenys_ + indeksas;
00631 }
00632
00639 T* erase(T* pirma, T* paskutine)
00640 {
00641     std::size_t pradzia = pirma - duomenys_;
00642     std::size_t pabaiga = paskutine - duomenys_;
00643
00644     if (pradzia > pabaiga || pabaiga > dydis_)
00645     {
00646         throw std::out_of_range("Erase range out of range");
00647     }
00648
00649     std::size_t kiekSalinti = pabaiga - pradzia;
00650
00651     for (std::size_t i = pradzia; i + kiekSalinti < dydis_; ++i)
00652     {
00653         duomenys_[i] = duomenys_[i + kiekSalinti];
00654     }
00655     dydis_ -= kiekSalinti;
00656     return duomenys_ + pradzia;
00657 }
00658
00663 void swap(Vector& kitas) noexcept
00664 {
00665     T* laikiniDuomenys = duomenys_;
00666     duomenys_ = kitas.duomenys_;
00667     kitas.duomenys_ = laikiniDuomenys;
00668
00669     std::size_t laikinasDydis = dydis_;
00670     dydis_ = kitas.dydis_;
00671     kitas.dydis_ = laikinasDydis;
00672
00673     std::size_t laikinaTalpa = talpa_;
00674     talpa_ = kitas.talpa_;
00675     kitas.talpa_ = laikinaTalpa;
00676 }
00677
00683 bool operator==(const Vector& kitas) const
00684 {
00685     if (dydis_ != kitas.dydis_)
00686     {
00687         return false;
00688     }
00689
00690     for (std::size_t i = 0; i < dydis_; ++i)
00691     {
00692         if (duomenys_[i] != kitas.duomenys_[i])
00693         {
00694             return false;
00695         }
00696     }
00697     return true;
00698 }
00704 bool operator!=(const Vector& kitas) const
00705 {
00706     return !(*this == kitas);
00707 }
00708
00715 void assign(std::size_t kiekis, const T& reiksme)
00716 {
00717     if (kiekis > talpa_)
00718     {
00719         delete[] duomenys_;
00720         duomenys_ = new T[kiekis];
00721         talpa_ = kiekis;

```

```

00722     }
00723     for (std::size_t i = 0; i < kiekis; ++i)
00724     {
00725         duomenys_[i] = reiksme;
00726     }
00727     dydis_ = kiekis;
00728 }
00729
00730 void assign(std::initializer_list<T> sarasas)
00731 {
00732     if (sarasas.size() > talpa_)
00733     {
00734         delete[] duomenys_;
00735         duomenys_ = new T[sarasas.size()];
00736         talpa_ = sarasas.size();
00737     }
00738
00739     std::size_t i = 0;
00740     for (const T& reiksme : sarasas)
00741     {
00742         duomenys_[i] = reiksme;
00743         ++i;
00744     }
00745     dydis_ = sarasas.size();
00746 }
00747
00748 void clear()
00749 {
00750     dydis_ = 0;
00751 }
00752
00753 Vector(const Vector& kitas)
00754 : duomenys_(nullptr), dydis_(kitas.dydis_), talpa_(kitas.talpa_)
00755 {
00756     if (kitas.talpa_ > 0)
00757     {
00758         duomenys_ = new T[talpa_];
00759
00760         for (std::size_t i = 0; i < kitas.dydis_; ++i)
00761         {
00762             duomenys_[i] = kitas.duomenys_[i];
00763         }
00764     }
00765 }
00766
00767 Vector& operator=(const Vector& kitas)
00768 {
00769     if (this != &kitas)
00770     {
00771         T* naujiDuomenys = nullptr;
00772
00773         if (kitas.talpa_ > 0)
00774         {
00775             naujiDuomenys = new T[kitas.talpa_];
00776
00777             for (std::size_t i = 0; i < kitas.dydis_; ++i)
00778             {
00779                 naujiDuomenys[i] = kitas.duomenys_[i];
00780             }
00781
00782             delete[] duomenys_;
00783
00784             duomenys_ = naujiDuomenys;
00785             dydis_ = kitas.dydis_;
00786             talpa_ = kitas.talpa_;
00787         }
00788         return *this;
00789     }
00790 }
00791
00792 Vector(Vector&& kitas) noexcept
00793 : duomenys_(kitas.duomenys_), dydis_(kitas.dydis_), talpa_(kitas.talpa_)
00794 {
00795     kitas.duomenys_ = nullptr;
00796     kitas.dydis_ = 0;
00797     kitas.talpa_ = 0;
00798 }
00799
00800 Vector& operator=(Vector&& kitas) noexcept
00801 {
00802     if (this != &kitas)
00803     {
00804         delete[] duomenys_;
00805
00806         duomenys_ = kitas.duomenys_;
00807         dydis_ = kitas.dydis_;
00808     }
00809 }

```

```

00839         talpa_ = kitas.talpa_;
00840
00841         kitas.duomenys_ = nullptr;
00842         kitas.dydis_ = 0;
00843         kitas.talpa_ = 0;
00844     }
00845     return *this;
00846 }
00847
00854 Vector& operator=(std::initializer_list<T> sarasas)
00855 {
00856     if (sarasas.size() > talpa_)
00857     {
00858         delete[] duomenys_;
00859         duomenys_ = new T[sarasas.size()];
00860         talpa_ = sarasas.size();
00861     }
00862
00863     std::size_t i = 0;
00864     for (const T& reiksme : sarasas)
00865     {
00866         duomenys_[i] = reiksme;
00867         i++;
00868     }
00869
00870     dydis_ = sarasas.size();
00871     return *this;
00872 }
00873 }
00874 };
00875
00876 #endif

```

8.5 Objektinis/Zmogus.h File Reference

```

#include <iostream>
#include <string>

```

Classes

- class [Zmogus](#)

Abstrakti bazine klase, skirta saugoti bendrus duomenis apie zmogu.

Macros

- #define [ZMOGUS_H](#)

8.5.1 Macro Definition Documentation

8.5.1.1 ZMOGUS_H

```
#define ZMOGUS_H
```

8.6 Zmogus.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #ifndef ZMOGUS_H
00003 #define ZMOGUS_H
00004 #include <iostream>
00005 #include <string>
00006
00015 class Zmogus
00016 {
00017     protected:
00021         std::string vardas_;
00025         std::string pavarde_;
00026
00027     public:
00031         Zmogus();

```

```
00032
00038     Zmogus(const std::string& vardas, const std::string& pavarde);
00039
00043     virtual ~Zmogus();
00044
00049     std::string getVardas() const;
00050
00055     std::string getPavarde() const;
00056
00061     void setVardas(const std::string& vardas);
00062
00067     void setPavarde(const std::string& pavarde);
00068
00073     virtual void read(std::istream& in) = 0;
00074
00079     virtual void print(std::ostream& out) const = 0;
00080 };
00081
00082 #endif // ZMOGUS_H
```

8.7 README.md File Reference

Index

- ~Studentas
 - Studentas, [23](#)
- ~Vector
 - Vector< T >, [30](#)
- ~Zmogus
 - Zmogus, [40](#)
- assign
 - Vector< T >, [31](#)
- at
 - Vector< T >, [31](#)
- back
 - Vector< T >, [31](#)
- begin
 - Vector< T >, [31](#)
- capacity
 - Vector< T >, [32](#)
- cbegin
 - Vector< T >, [32](#)
- cend
 - Vector< T >, [32](#)
- clear
 - Vector< T >, [32](#)
- data
 - Vector< T >, [32](#)
- emplace
 - Vector< T >, [32](#)
- emplace_back
 - Vector< T >, [33](#)
- empty
 - Vector< T >, [33](#)
- end
 - Vector< T >, [33](#)
- erase
 - Vector< T >, [33](#), [34](#)
- front
 - Vector< T >, [34](#)
- getEgzaminas
 - Studentas, [24](#)
- getPavarde
 - Zmogus, [40](#)
- getPazymiai
 - Studentas, [24](#)
- getRezultatas
 - Studentas, [24](#)
- getSunaikintuObjektuKiekis
 - Studentas, [24](#)
- getVaras
 - Zmogus, [40](#)
- insert
 - Vector< T >, [34](#), [35](#)
- max_size
 - Vector< T >, [35](#)
- Objektinis Directory Reference, [19](#)
- Objektinis/Studentas.h, [43](#)
- Objektinis/Vector.h, [44](#), [45](#)
- Objektinis/Zmogus.h, [52](#)
- operator!=
 - Vector< T >, [35](#)
- operator<<
 - Studentas, [26](#)
- operator>>
 - Studentas, [26](#)
- operator=
 - Studentas, [24](#), [25](#)
 - Vector< T >, [36](#)
- operator==
 - Vector< T >, [36](#)
- operator[]
 - Vector< T >, [37](#)
- pavarde_
 - Zmogus, [41](#)
- pop_back
 - Vector< T >, [37](#)
- pridetiPazymi
 - Studentas, [25](#)
- print
 - Studentas, [25](#)
 - Zmogus, [40](#)
- push_back
 - Vector< T >, [37](#)
- read
 - Studentas, [25](#)
 - Zmogus, [40](#)
- README.md, [53](#)
- reserve
 - Vector< T >, [37](#)
- resize
 - Vector< T >, [37](#), [38](#)
- setEgzaminas

- Studentas, 25
- setPavarde
 - Zmogus, 40
- setPazymiai
 - Studentas, 26
- setRezultatas
 - Studentas, 26
- setVardas
 - Zmogus, 41
- shrink_to_fit
 - Vector< T >, 38
- size
 - Vector< T >, 38
- Studentas, 21
 - ~Studentas, 23
 - getEgzaminas, 24
 - getPazymiai, 24
 - getRezultatas, 24
 - getSunaikintuObjektuKiekis, 24
 - operator<<, 26
 - operator>>, 26
 - operator=, 24, 25
 - pridetiPazymi, 25
 - print, 25
 - read, 25
 - setEgzaminas, 25
 - setPazymiai, 26
 - setRezultatas, 26
 - Studentas, 23, 24
- Studentas.h
 - STUDENTAS_H, 43
- STUDENTAS_H
 - Studentas.h, 43
- Studentu pazymiu skaiciavimo programa, 1
- swap
 - Vector< T >, 38
- vardas_
 - Zmogus, 41
- Vector
 - Vector< T >, 29, 30
- Vector< T >, 27
 - ~Vector, 30
 - assign, 31
 - at, 31
 - back, 31
 - begin, 31
 - capacity, 32
 - cbegin, 32
 - cend, 32
 - clear, 32
 - data, 32
 - emplace, 32
 - emplace_back, 33
 - empty, 33
 - end, 33
 - erase, 33, 34
 - front, 34
 - insert, 34, 35
 - max_size, 35
 - operator!=, 35
 - operator=, 36
 - operator==, 36
 - operator[], 37
 - pop_back, 37
 - push_back, 37
 - reserve, 37
 - resize, 37, 38
 - shrink_to_fit, 38
 - size, 38
 - swap, 38
 - Vector, 29, 30
- Vector.h
 - VECTOR_H, 44
- VECTOR_H
 - Vector.h, 44
- Zmogus, 38
 - ~Zmogus, 40
 - getPavarde, 40
 - getVardas, 40
 - pavarde_, 41
 - print, 40
 - read, 40
 - setPavarde, 40
 - setVardas, 41
 - vardas_, 41
 - Zmogus, 39
- Zmogus.h
 - ZMOGUS_H, 52
- ZMOGUS_H
 - Zmogus.h, 52